

# FINAL YEAR PROJECT PROPOSAL

Department of Computer Science  
National University of Computer and Emerging Sciences

---

## PROJECT A.E.G.I.S.

**Adaptive Execution and Goal-oriented Intelligence System**

---

### Project Team Members

Hira Khalid - 23L-2594  
Doureesha Batool - 23L-2651  
Hashir Rauf - 23L-2572

---

**Domain:** Artificial Intelligence · Human-Computer Interaction · Agentic AI · NLP · Operating Systems

## 1. Abstract

---

Final-year students routinely lose substantial working time to a narrow but recurring problem: after a day or more away from a project, they cannot quickly reconstruct what they were doing, where their files are, or what they had already decided. Contemporary AI-powered assistants - including Apple Siri, Amazon Alexa, and the newer generation of "computer-use" agents now shipped by Microsoft, Anthropic, Google, and OpenAI - execute explicit commands or operate a screen directly, but none are built to solve this specific problem: giving an individual, privacy-conscious student a transparent, bounded memory of their own work.

This proposal presents AEGIS (Adaptive Execution and Goal-oriented Intelligence System), a bounded, opt-in desktop copilot for final-year students. Rather than observing an entire computer, AEGIS operates only on workspaces - folders and browser tabs the student explicitly chooses to share - and provides four concrete capabilities: resuming a work session after a gap, tidying a messy folder with reversible AI-suggested grouping, pulling cited notes together from a handful of documents, and noticing repeated manual patterns to offer (never impose) automation.

The research contribution is threefold: (1) a small, auditable context model - the "What AEGIS Knows" panel - that fuses only explicitly-shared file, browser-tab, and session-note signals, in contrast to the ambient, ecosystem-wide context retrieval pursued by Microsoft Recall and Google Gemini; (2) a risk-tiered, permission-explicit action model that excludes deletion and outbound communication entirely in its first version, evaluated against a human-in-the-loop approval UX rather than mere guardrail existence; and (3) an empirical evaluation methodology - a three-phase pilot, lab, and field-bridged design - built specifically to measure the one feature (session resume) that a single-sitting lab study structurally cannot test. A competitive review of the 2026 desktop-copilot landscape and a targeted literature audit (Sections 4 and 9) ground these choices in what is, and is not, already solved by industry and prior research.

## 2. Background and Motivation

---

### 2.1 The Problem, Stated Narrowly

When a final-year student returns to their FYP or thesis work after a day or more away, they lose real time re-finding files, re-reading prior research, and reconstructing what they had already decided and what to do next. The goal of AEGIS is to cut that reconstruction time by a measurable margin, before the student can get back to substantive work. This is deliberately narrower than "reduce cognitive load in knowledge work" in general: research by González and Mark (2004; 2022) establishes that fragmented attention and task-switching costs are real and persistent across the wider knowledge-work population, but AEGIS targets one well-defined slice of that problem - a single person, a single recurring pain point, and one feature (Section 8.1) built to answer it directly.

## 2.2 Limitations of Current AI Assistants

The first generation of AI-powered desktop assistants - Cortana, Siri, Google Assistant - represented a meaningful step towards natural language interaction with computing environments, but shared architectural limitations: reactive command execution, shallow contextual awareness, no long-term memory across sessions, no multi-step planning, and limited desktop integration.

As of mid-2026, this landscape has changed substantially, and the change matters for how AEGIS should be scoped. Microsoft's Copilot Studio Computer Use reached general availability on 13 May 2026, giving agents direct browser, screen, and keyboard control; Anthropic's Claude Computer Use and Cowork agent, OpenAI's ChatGPT Agent Mode and Atlas browser, and Google's Gemini Computer Use tool all now offer a mature, vision-based "the AI operates a computer" capability. Microsoft Recall provides a local, opt-in, photographic-memory search over on-screen history. The mechanical capability that early AEGIS drafts treated as a research contribution - an AI that can see and act on a desktop - is therefore no longer novel on its own; the detailed comparison is given in Section 4.

## 2.3 What Remains Genuinely Open

Despite this, every current industry player is built either for enterprise-scale workflow automation or for ecosystem-locked consumer convenience (Windows, iOS, Google Workspace). None is built as a small, transparent, individually-owned assistant scoped to a single student's own bounded workspace, with no ambient collection outside what the student has explicitly chosen to share. That gap - not the underlying computer-use mechanism - is where AEGIS is positioned to contribute, and it is the basis for the narrowed, four-feature design described in Section 8.

## 3. Problem Statement

---

Despite the existence of increasingly capable AI-powered tools, an individual student still has no system that can:

1. Reconstruct, within seconds, what they were working on before a gap in activity - which files, which browser tabs, and what they intended to do next.
2. Tidy a messy, explicitly-chosen folder without risking silent, irreversible file movement.
3. Pull together notes and citations from a small, explicitly-chosen set of documents without the system reading anything beyond what was pointed at.
4. Notice a manually repeated file-management pattern and offer - never impose - automation, with every run logged and undoable.
5. See, at any time, an exact and complete account of what the system currently knows, with the ability to remove any item.

The core research problem may therefore be stated as follows: can a small, bounded, fully auditable desktop copilot - one that only ever sees what a student explicitly shares - measurably reduce the time and effort of resuming interrupted academic work, while remaining safe, transparent, and trustworthy enough that students choose to keep it installed?

## 4. Competitive Landscape and Research Gap

### 4.1 What Existing Industry Players Are Building (as of mid-2026)

A structured review of the six most relevant industry players was conducted and independently fact-checked against current sources. The verified findings are summarised below; two claims present in earlier drafts of this review required correction and are flagged accordingly.

| Company    | Product / Capability               | Verified Status                                                                                                                                                                                                                                                                                                                                                                          |
|------------|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Microsoft  | Copilot Studio Computer Use        | GA 13 May 2026. Vision-based UI control. Supports native Windows frameworks and browser sites only - Electron apps (VS Code, Slack, Discord), Java enterprise software, and Citrix apps remain unsupported (desktop-task success ~35% vs. ~80% for web tasks).                                                                                                                           |
| Microsoft  | Recall                             | Broadly available since ~April 2025 (not a new mid-2026 feature). Local, opt-in, on-device screenshot search; does not transmit data to Microsoft.                                                                                                                                                                                                                                       |
| Microsoft  | Copilot Tasks                      | Cloud-hosted background agent, isolated per-task VM. Remains in research preview, not general availability, as of writing.                                                                                                                                                                                                                                                               |
| Anthropic  | Claude Computer Use / Cowork       | Computer Use in research preview from March 2026; Cowork reached GA across paid plans by April 2026. Sandboxed execution with real-time safety classifiers.                                                                                                                                                                                                                              |
| OpenAI     | ChatGPT Agent Mode / Atlas browser | Operator retired as a standalone product August 2025; capability now powers Agent Mode and the Atlas browser (launched October 2025). The often-cited 87% WebVoyager figure belongs to the underlying CUA model's 2025 benchmark, not a verified Atlas-specific score.                                                                                                                   |
| Google     | Gemini Computer Use                | API-level tool with an explicit safety_decision field (allowed / require_confirmation / blocked). Project Mariner, Google's dedicated browser-agent prototype, was discontinued 4 May 2026 and folded into Gemini Agent and Chrome.                                                                                                                                                      |
| Apple      | Apple Intelligence / Siri          | On-device, personal-context-driven. Note: Apple is not simply avoiding autonomous automation, as earlier competitive drafts assumed - the Passwords app now autonomously navigates and updates credentials, and "Autonomous Transactions" is reported for late 2026/2027. The real distinction is architectural (declarative App Intents vs. raw UI-pixel control), not one of ambition. |
| Perplexity | Comet Browser / Comet Enterprise   | Agentic, search-centric browser; Enterprise edition (March 2026) ships MDM deployment on macOS/Windows, signalling a shift toward managed, organisation-scale automation rather than an individually-owned assistant.                                                                                                                                                                    |

One additional player, absent from earlier competitive drafts, is directly relevant given AEGIS's specific memory-and-context framing: Rewind.ai, a local-memory, persistent-timeline product

that predates and closely parallels Microsoft Recall. It is, in important respects, a closer analog to AEGIS's "memory" pitch than any of the six companies above, and is acknowledged here to preempt the obvious comparison.

## 4.2 Research Gaps, Re-tested Against 2025-2026 Literature

The five research gaps first identified for AEGIS were independently re-tested against literature published through mid-2026. None are fully closed, but two required meaningful narrowing to remain defensible; the revised statements are given below.

**G1 - Bounded, Auditable Context Integration:** No existing system unifies explicitly student-selected file, browser-tab, and session-note signals into a single retrieval corpus for autonomous desktop task execution, as opposed to ambient context-capture tools (e.g., Contexty, arXiv 2604.11067) or deep GUI-action agents (e.g., UFO2, arXiv 2504.14603) that assume unrestricted access to the whole desktop.

**G2 - Desktop-Coupled Episodic Memory:** General-purpose episodic memory for LLM agents is no longer an open problem - frameworks such as Mem0, A-MEM, and Letta (2025) now provide this off the shelf, and IronEngine (2026) already pairs desktop GUI automation with a persistent memory lifecycle. What remains open is memory natively coupled to a desktop copilot's own action-taking loop, tied to OS-level context (files opened, tabs tagged) rather than conversational turns alone, and made fully visible and editable to the end user.

**G3 - Cognitive-Load Evaluation for Desktop Automation Specifically:** NASA-TLX and related instruments are now used for LLM coding-assistant evaluation (see the systematic review at arXiv 2507.03156) and ambient clinical-AI tools, but remain absent for OS-level, multi-application agentic automation of the kind AEGIS proposes.

**G4 - Human-in-the-Loop Approval UX, Not Guardrail Existence:** Multiple 2025-2026 systems (AGrail, arXiv 2502.11448; OSGuard, arXiv 2606.15034; AgentDoG, arXiv 2601.18491) now provide automated risk detection and safety benchmarking for OS-level agents, so the gap can no longer be stated as guardrails being broadly "inadequately addressed." What none of these evaluate is a human-in-the-loop approval interface integrated with a copilot's own memory and planning loop, or how approval friction trades off against user-perceived cognitive load and trust - the specific angle AEGIS's Safety Layer and evaluation target.

**G5 - Proactive vs. Reactive Assistance Beyond the Single-IDE Case:** CHI 2025 work ("Assistance or Disruption?", arXiv 2502.18658; "Need Help?") has rigorously studied this trade-off for code-editor copilots specifically, finding persistent proactive suggestions often rated as distracting. This question remains open for general-purpose desktop copilots spanning file management, browsing, and document work, where interruption cost and task-switching context differ from a single-IDE setting.

## 5. Research Questions

---

**RQ1 [Session Resumption]:** Does a bounded, auditable summary of recently-touched files, tagged tabs, and a student's own closing note measurably reduce the time and effort required to resume an interrupted academic work session, compared to unaided manual reconstruction?

**RQ2 [Reversible Automation Trust]:** Does a preview-then-approve-then-undo interaction pattern for file organisation and repeated-task automation increase user trust and perceived control, without introducing unacceptable friction for low-risk actions?

**RQ3 [Cognitive Load]:** Does interaction with AEGIS measurably reduce cognitive workload - assessed with Raw NASA-TLX at the task level - for the specific tasks of session resumption, folder organisation, and multi-document note synthesis, compared to unaided baseline interaction?

**RQ4 [Grounded Multi-Document Synthesis]:** Can a small, explicitly-scoped retrieval pipeline produce citation-grounded summaries across a handful of student-selected documents with acceptable accuracy and acceptably low hallucination, when citation metadata (DOI/Crossref) is available?

## 6. Research Objectives

- 
6. O1: To design and implement a bounded, opt-in workspace model in which the system tracks only files and browser tabs explicitly added by the student, with a persistent, user-visible "What AEGIS Knows" transparency panel.
  7. O2: To build the Pick Up Where You Left Off feature: a session-resume summary combining recently-changed workspace files, tagged browser tabs, saved session notes, and a one-click reopen action for each item.
  8. O3: To build the Tidy Up a Folder feature: AI-suggested, preview-first file grouping with full approval and a working group-level undo.
  9. O4: To build the Pull Together Notes feature: citation-grounded synthesis across a small, explicitly-selected set of documents, producing a new file that never overwrites existing work.
  10. O5: To build the Notice a Repeated Task feature: heuristic detection of a manually repeated file-management pattern (occurring three or more times), surfaced as an editable, deletable, opt-in automation rule.
  11. O6: To implement a risk-tiered action model that excludes file deletion and all outbound communication from the first version, and requires explicit approval for any file move or rename.
  12. O7: To evaluate the system through a three-phase study (Section 11) measuring session-resume effectiveness, task-level cognitive workload, and user trust/control, including a field-bridged design specifically for the resume feature.

## 7. Proposed Methodology

---

## 7.1 Research Design

This project adopts a Design Science Research (DSR) methodology (Hevner et al., 2004), proceeding through three iterative phases: (1) problem identification, competitive review, and requirements analysis; (2) design, prototyping, and iterative refinement of the four bounded features; and (3) empirical evaluation through the three-phase study described in Section 11.

## 7.2 Phase 1 - Requirements and Feasibility Analysis

This phase encompasses the following activities:

- Comparative review of existing AI desktop assistants and copilot systems (Section 4).
- Feasibility verification of each of the four proposed features against available libraries, prior open-source work, and known technical pitfalls (Section 9).
- Definition of the exact context model - what is collected, what is explicitly not (Section 8.3) - before any implementation begins.
- Elicitation of the risk-tiered action rules (Section 8.4).

## 7.3 Phase 2 - System Design and Prototyping

The four features will be built in the order given in Section 8.1, each shipped to a working, demoable state before the next is started. Development will follow agile principles with two-week sprint cycles. Feature 1 (session resume) is scoped to file-system and local-database tracking only for the first working version; browser-tab tagging via an extension is treated as a stretch goal, not a v1 requirement (Section 9.5).

## 7.4 Phase 3 - Empirical Evaluation

Evaluation follows the three-phase design given in full in Section 11: a formative usability pilot, a single-session lab comparison for tasks that fit one sitting, and a field-bridged evaluation specifically for the session-resume feature, which cannot be validly tested within a single lab session. Ethical approval will be obtained from the university research ethics committee prior to any participant recruitment.

# 8. System Design

---

## 8.1 Design Principle: Small and Bounded, Not Everything

The central design decision of this project is that AEGIS does not watch the whole computer. It only knows about workspaces - folders and browser tabs the student chooses to share with it. This is what makes the project achievable within two semesters, easy for a student to trust on first use, and structurally different from ambient tools like Microsoft Recall or Copilot, which are built to see everything by default.

A workspace is created on purpose: the student right-clicks a folder (e.g. D:\FYP\AEGIS) and selects "Add as AEGIS workspace"; nothing outside that folder is touched. Browser tabs are added one at a time via a small browser extension that tags a tab to a specific workspace; there

is no silent background tracking of browsing activity. Everything the system knows is visible on one screen - the "What AEGIS Knows" panel - listing every file, tab, and saved note currently tracked, each with its own delete control.

## **8.2 The Four Features**

Built in this order:

### **Feature 1 - Pick Up Where You Left Off (build first)**

The system's primary, most directly measured feature. Triggered by opening the AEGIS window and selecting Resume, or typing "where was I." It gathers the five most recently changed files in the workspace, the last three saved session notes, and whatever the student said they were about to do next when they last closed the app; browser-tab tracking is a stretch goal for later iterations (Section 9.5), not a requirement of the first working version. It shows one summary card every time: last active date, files worked in, tabs open (once the extension ships), the last note, and what the student said they were about to do next. Every file or tab has a one-click reopen. When the workspace is closed, or after a period of inactivity, AEGIS asks one optional question - "What were you about to do next?" - and shows the answer back verbatim next time, with no AI interpretation involved.

### **Feature 2 - Tidy Up a Folder (build second)**

Triggered by pointing AEGIS at a messy folder inside the workspace and asking it to organise it. AEGIS reads file names and a first-page text extract of each document, then proposes groupings as a preview - nothing moves until the student approves the whole plan or accepts/rejects items individually. A single "Undo last organise" action restores every file to its original location.

### **Feature 3 - Pull Together Notes From a Few Documents (build third)**

Triggered by a request such as "pull the citations from the PDFs in /refs into a references section" or "tell me where these four papers agree and disagree." Only the files the student explicitly points to are read - never the whole workspace. A new file is always created; existing files are never overwritten. Citations follow APA 7th edition for the first version, and are reliably accurate only where DOI or Crossref metadata is discoverable (Section 9.3); the new file opens immediately for review.

### **Feature 4 - Notice a Repeated Task and Offer to Handle It (build last)**

If AEGIS observes the same manual pattern occurring three or more times within a workspace - for example, downloading a PDF and always moving it to the same folder under the same naming convention - it asks a single question: "Want me to do this automatically next time?" It never acts on a pattern without asking first. Accepting creates a rule that appears in a plain, editable, deletable list; every automatic run is logged and can be undone the same way as the folder-tidy feature.



### 8.3 Context Model - What the System Is Allowed to Look At

This table replaces the vague notion of "multi-source desktop context" with an exact, auditable list. If a signal is not in this table, the first version does not collect it.

| Source                 | Collected                                                                             | Explicitly Not Collected                                                    |
|------------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Workspace files        | File name, folder path, last-changed time, a short first-page text extract for search | Full file contents tracked continuously; anything outside the chosen folder |
| Browser (stretch goal) | Tab title and link, only for tabs the student has tagged                              | Full page content, background browsing history, any untagged tab            |
| Session notes          | The short note about what the student was about to do, with a timestamp               | Keystrokes, screen recordings, audio - none of this is part of this design  |
| Rules                  | Automation rules the student has explicitly agreed to                                 | Any rule the system creates or runs without the student's consent           |

Everything is stored locally in a small database on the student's own laptop. Nothing is synced to the cloud by default. If a cloud LLM is used for the notes-synthesis feature, it sees only the specific files the student pointed to - never the workspace list as a whole.

### 8.4 Action Model - What the System Is Allowed to Do

| Action                                      | Risk Level         | Rule                                                                                           |
|---------------------------------------------|--------------------|------------------------------------------------------------------------------------------------|
| Read a file or tab already in the workspace | Safe               | Happens quietly, no confirmation required                                                      |
| Create a new file (notes feature)           | Safe               | No confirmation required, but the file is clearly marked as AEGIS-created and opens for review |
| Move or rename files                        | Needs confirmation | Shown as a preview first; can be undone as a group for 24 hours                                |
| Delete a file                               | Not permitted      | Excluded entirely from the first version                                                       |
| Send anything outside the app (e.g. email)  | Not permitted      | Excluded entirely from the first version - no outgoing messages of any kind                    |
| Add or remove a workspace or tagged tab     | Safe, student-only | Only the student can perform this action; the system never adds one on its own                 |

Excluding deletion and outbound communication is a deliberate scope decision, not an oversight: these are the two hardest categories of action to make trustworthy, so the first version avoids them entirely rather than attempting to solve trust, safety, and core functionality simultaneously.

## 9. Feasibility and Technical Risk

---

Each feature was independently checked against available tooling, comparable prior work, and known failure modes before being committed to the build plan.

### 9.1 Feature 1 - Pick Up Where You Left Off

**Risk:** Low-to-medium risk

No AI/ML sits in the critical path. File tracking uses mature, well-documented libraries (e.g. watchdog for Windows file-system events); a periodic scan sorted by modified time is more robust than a persistent watcher for this use case. Comparable tools exist (Session Buddy, Workona, Arc Boosts for tab management) but none combine file, tab, and note state into one resume card, which keeps the feature genuinely differentiated. The one caveat: ReadDirectoryChangesW is unreliable over network drives and can silently drop events on very large folders, which should be tested early.

## 9.2 Feature 2 - Tidy Up a Folder

**Risk:** Medium risk

The extract-classify-preview-approve-move-undo pipeline is well established, and directly comparable open-source tools exist (e.g. ai-file-sorter, Local-File-Organizer) confirming the approach is buildable by a small team. PyMuPDF is a fast, practical choice for first-page text extraction. Scanned or image-only PDFs require OCR and are explicitly out of scope for the first version; undo must be implemented as a transactional move-log, not a best-effort retry.

## 9.3 Feature 3 - Pull Together Notes

**Risk:** Medium-to-high risk

The highest-risk feature. A small, hand-rolled retrieval pipeline (chunk, embed, retrieve, synthesise) is appropriate at this scale - a full framework such as LangChain adds overhead not justified for a handful of documents. Reliable APA formatting depends on citeproc-py plus Crossref metadata lookup, which works well only when a DOI is discoverable; documents without one need a manual-correction fallback. Every synthesis claim must be grounded in retrieved text with a page/section citation to bound hallucination risk. Scope for v1: DOI-available documents only, with broader coverage treated as a stretch goal.

## 9.4 Feature 4 - Notice a Repeated Task

**Risk:** Medium risk

Formal sequence-mining algorithms are built for large-scale event logs and are unnecessary for a single user's sparse action stream; a simple heuristic tuple-match (source folder, destination folder, filename pattern) with a rolling counter is easier to build, explain, and keep transparent. Three occurrences is a small sample, so the match definition must be tight enough to avoid noisy false positives - this is a UX tuning problem more than an algorithmic one, and needs iteration time in the schedule.

## 9.5 Scope-Surface Warning

No comparable capstone or final-year project was found that successfully shipped a desktop application, a browser extension, and three separate LLM-integrated features within a single project cycle. The pattern across similar existing tools is that a small team ships one feature well, not several. The browser extension required for full tab-tracking in Feature 1 introduces a distinct technical stack - extension packaging, manifest and store review quirks, and cross-process messaging to the desktop app - that the other three features do not share. Accordingly, tab-tracking is treated as a stretch goal rather than a v1 requirement (Section 8.2); the first

working version of Feature 1 uses file-system and local-database tracking only. This preserves the three AI-differentiated features, which are the stronger material for evaluation and defense, while removing one disproportionately costly technical surface.

## 10. Novel Contributions

---

Given the competitive landscape in Section 4, this project's contribution is deliberately not the underlying computer-use mechanism, which is now well-solved by major industry players. The contribution instead lies in the following:

**C1 - Bounded, Auditable Context Model:** An opt-in workspace model in which every piece of context the system holds is traceable to a specific, student-initiated sharing action, made fully visible through the "What AEGIS Knows" panel - in direct contrast to the ambient, opt-out-style context collection pursued by Recall, Gemini, and Copilot.

**C2 - Session-Resume as a First-Class, Independently Evaluated Feature:** A dedicated resume feature evaluated with a study design (Section 11) built specifically around the fact that task-resumption cannot be validly tested in a single lab sitting - addressing a known, unresolved methodological gap in the interruption-recovery literature (Iqbal and Horvitz, 2007; Czerwinski et al., 2004).

**C3 - Reversible-by-Default Automation:** A risk-tiered action model that excludes the two hardest-to-trust action categories (deletion, outbound communication) entirely from the first version, and requires preview-and-undo for every action that touches the file system - a narrower, more conservative safety design than the guardrail-and-benchmark approach taken by current industry safety research (Section 4.2, G4).

**C4 - Grounded, Citation-Bound Multi-Document Synthesis at Small Scale:** A retrieval pipeline scoped deliberately small (a handful of student-selected documents, not an open corpus) with every synthesised claim traceable to a source passage - prioritising verifiability over breadth.

**C5 - A Task-Resumption-Appropriate Evaluation Methodology:** A three-phase evaluation design (pilot, single-session lab, field-bridged) that matches the evaluation instrument to what each feature can actually be validly tested for within a one-semester timeline (Section 11).

## 11. Evaluation Methodology

---

A single 20-30 participant controlled study with one round of NASA-TLX was considered and rejected. It has two structural problems confirmed during methodology review: it provides no stated statistical power justification, and - more importantly - there is no credible way to simulate a student "returning after a day or more away" within a single lab session. No prior task-resumption study (Iqbal and Horvitz, 2007; Czerwinski, Horvitz, and Wilhite, 2004; Jeuris and Bardram, 2016) has ever attempted to fake a multi-day gap in one sitting; they instead use

field or diary methodology across real sessions, or simulate fragmentation via repeated short interruptions rather than elapsed time. Evaluation is therefore restructured into three lighter, sequenced phases.

### 11.1 Phase 1 - Formative Usability Pilot

**Participants:** 6-8, justified by Nielsen's five-user heuristic for formative, qualitative usability testing

Think-aloud usability testing across all four features. The goal is fixing interaction problems before the main study; no inferential statistics are required at this stage.

### 11.2 Phase 2 - Single-Session Lab Comparison

**Participants:** 12-16, within-subjects, counterbalanced

Restricted to tasks that genuinely fit one sitting: folder organisation (Feature 2) and multi-document summarise/cite (Feature 3). Metrics: task completion time, number of manual interactions, task success rate, Raw NASA-TLX administered per task rather than once at the end of the session, the System Usability Scale, and a validated trust instrument (the 8-item Trust Scale for AI Context, or the Human-Generative AI Trust Scale) in place of a from-scratch safety-acceptance questionnaire. The target effect size and resulting statistical power will be stated explicitly, and the study will be framed honestly as exploratory/pilot-scale rather than a fully powered summative experiment.

### 11.3 Phase 3 - Field-Bridged Resume Evaluation

**Participants:** 10-15, a subset of Phase 2 participants

Following the Iqbal and Horvitz / Czerwinski precedent directly: participants use AEGIS for real coursework across a natural 2-4 day gap between two logged sessions, with a short diary entry captured at the moment of resumption plus a brief follow-up interview. This is the only credible way to evaluate Pick Up Where You Left Off, and is treated in the literature as an open methodological problem, not a shortcut.

### 11.4 Analysis Approach

Given the smaller per-phase sample sizes, paired non-parametric tests (Wilcoxon signed-rank) will be used, with effect sizes reported alongside confidence intervals rather than relying on  $p < 0.05$  as the sole basis for claims. Interview and diary data will be analysed using thematic analysis. The evaluation is framed throughout as a mixed-methods, effect-size-oriented pilot evaluation, consistent with current practice in LLM/agent human-centred evaluation (e.g. the CHI HEAL workshop series).

### 11.5 Ethical Considerations

This design sits in low-risk, expedited-review territory: voluntary adult participants, no deception, and no sensitive data provided the documents used in Phase 2/3 are drawn from participants' own coursework. Submission to the institutional ethics process will be initiated early in the timeline regardless, since even minimal-risk studies typically require a documented

submission, and approval timelines can otherwise consume a disproportionate share of a single semester.

## 12. Technology Stack

Selections below reflect the feasibility review in Section 9 and favour small, well-documented libraries over heavier frameworks, given the bounded scope of each feature.

| Layer                            | Selection                                                               | Notes                                                                                                                                                                        |
|----------------------------------|-------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Desktop application              | Python-based desktop shell (framework TBD in Phase 1)                   | Windows-first for the first version                                                                                                                                          |
| Browser extension (stretch goal) | Manifest V3, WXT/Plasmo/CRXJS boilerplate                               | Local HTTP server on localhost for the extension-to-desktop bridge, in place of native messaging, which is markedly more finicky to set up and debug                         |
| File-system tracking             | watchdog / watchfiles, periodic mtime scan as fallback                  | ReadDirectoryChangesW is unreliable over network drives - tested early                                                                                                       |
| PDF / document text extraction   | PyMuPDF (primary), pypdf (fallback)                                     | First-page extraction only for v1; scanned/image PDFs explicitly out of scope                                                                                                |
| Citation formatting              | citeproc-py (CSL-based APA 7), habanero (Crossref lookup)               | Reliable only where a DOI is discoverable; manual-correction fallback otherwise                                                                                              |
| Retrieval / synthesis            | Hand-rolled chunk-embed-retrieve pipeline                               | Small document sets do not justify a full RAG framework such as LangChain                                                                                                    |
| Local storage                    | SQLite                                                                  | All context and audit-log data stored locally by default                                                                                                                     |
| LLM                              | Cloud API (cost-trivial at this scale) with a local-model fallback path | e.g. GPT-4o-mini class pricing for classification/synthesis; Ollama-hosted small models as a local-processing option consistent with the privacy commitments in Section 14.2 |
| Repeated-task detection          | Heuristic tuple-match with rolling counters                             | Formal sequence-mining libraries (PrefixSpan, PAMI) are unnecessary at this scale                                                                                            |

## 13. Project Timeline

| Phase                                 | Activities                                                                                         | Duration   | Milestone                                                      |
|---------------------------------------|----------------------------------------------------------------------------------------------------|------------|----------------------------------------------------------------|
| Phase 1<br>Requirements & Feasibility | Competitive review; feasibility verification per feature; context/action model definition          | Weeks 1-4  | Requirements specification; finalised context and action model |
| Phase 2<br>System Design              | Architecture design; database schema; UI/UX design for workspace panel and "What AEGIS Knows" view | Weeks 5-8  | System design document                                         |
| Phase 3<br>Features 1 & 2             | Pick Up Where You Left Off (file/DB tracking) and Tidy Up a Folder, unit and integration tested    | Weeks 9-16 | Two working, demoable features                                 |

|                                  |                                                                                                                  |             |                                                         |
|----------------------------------|------------------------------------------------------------------------------------------------------------------|-------------|---------------------------------------------------------|
| Phase 4<br>Features 3 & 4        | Pull Together Notes and Notice a Repeated Task, unit and integration tested                                      | Weeks 17-24 | Complete four-feature prototype                         |
| Phase 5<br>Evaluation            | Ethics approval; Phase 1-3 evaluation (Section 11) - formative pilot, lab comparison, field-bridged resume study | Weeks 25-32 | Evaluation dataset; raw results across all three phases |
| Phase 6<br>Analysis and Write-up | Statistical and thematic analysis; dissertation writing; revision; submission                                    | Weeks 33-36 | Final dissertation; project poster; demo video          |

Browser-tab tagging for Feature 1 (Section 8.2, Section 9.5) is scheduled only if Phase 3 completes ahead of its allotted four weeks; it is not on the critical path.

## 14. Limitations, Ethical Considerations, and Future Work

---

### 14.1 Limitations

- Platform Scope: The prototype targets Windows only; cross-platform generalisation is future work.
- Deliberately Narrow Action Set: File deletion and all outbound communication are excluded from the first version by design (Section 8.4), which limits the scope of automation the system can offer.
- LLM Dependency: Feature 3's synthesis quality depends on the underlying LLM and on DOI/Crossref metadata availability; citation accuracy is not guaranteed for documents lacking a discoverable DOI (Section 9.3).
- Evaluation Sample Sizes: The three-phase design (Section 11) uses smaller, phase-appropriate samples rather than one large cohort; results are reported as exploratory and effect-size-oriented rather than as a fully powered summative claim.
- Feature 1 Scope: Browser-tab tracking is a stretch goal, not a guaranteed v1 deliverable (Section 9.5); the resume feature may ship with file/DB tracking only.

### 14.2 Ethical Considerations and Privacy

AEGIS operates in close proximity to sensitive user data. The following commitments guide system design and evaluation:

- Data Minimisation: The system tracks only what a student has explicitly added to a workspace (Section 8.3); nothing outside a chosen folder or tagged tab is touched.
- Local Processing Priority: All context and audit-log data is stored locally by default; where a cloud LLM is used, it sees only the specific documents pointed to for a given request, never the full workspace listing.
- Informed Consent: All study participants receive clear, jargon-free explanations of what is collected, how it is used, and their right to withdraw at any point.
- Transparency by Design: The "What AEGIS Knows" panel (Section 8.1) and the plan-preview-before-action pattern (Section 8.4) make the system's knowledge and intended actions fully visible before anything happens.

- Safety by Design, Not Afterthought: The exclusion of deletion and outbound communication (Section 8.4) is a first-class design decision made before implementation, not a policy layered on afterward.
- Audit and Accountability: Every executed, approved, or undone action is logged in a user-accessible audit trail.

### 14.3 Future Work

- Browser-tab tracking for Feature 1, if not completed within the core timeline (Section 9.5).
- Extension to macOS and Linux.
- Controlled introduction of higher-risk actions (e.g. supervised outbound communication) once trust and safety findings from Phase 2/3 evaluation are available.
- OCR support for scanned/image-only documents in Features 2 and 3.
- Longitudinal study of rule accumulation and personalisation quality (Feature 4) over an extended usage period beyond this project's timeline.

## 15. References

- 
13. AGrail: A Lifelong Agent Guardrail with Effective and Adaptive Safety Detection. *arXiv preprint arXiv:2502.11448*, 2025.
  14. AgentDoG: A Diagnostic Guardrail Framework for AI Agent Safety and Security. *arXiv preprint arXiv:2601.18491*, 2026.
  15. AgentOS: From Application Silos to a Natural Language-Driven Data Ecosystem. *arXiv preprint arXiv:2603.08938*, 2026.
  16. Assistance or Disruption? Exploring and Evaluating the Design and Trade-offs of Proactive AI Programming Support. *Proceedings of CHI 2025 / arXiv preprint arXiv:2502.18658*.
  17. Contexty: Capturing and Organizing In-situ Thoughts for Context-Aware AI Support. *arXiv preprint arXiv:2604.11067*, 2026.
  18. Czerwinski, M., Horvitz, E., & Wilhite, S. (2004). A diary study of task switching and interruptions. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (pp. 175–182). ACM.
  19. González, V. M., & Mark, G. (2004). 'Constant, constant, multi-tasking craziness': Managing multiple working spheres. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (pp. 113–120). ACM.
  20. Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105.
  21. Iqbal, S. T., & Horvitz, E. (2007). Disruption and recovery of computing tasks: Field study, analysis, and directions. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (pp. 677–686). ACM.
  22. Jeuris, S., & Bardram, J. E. (2016). Dedicated workspaces: Faster resumption times and reduced cognitive load in sequential multitasking. *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*.
  23. Need Help? Designing Proactive AI Assistants for Programming. *Proceedings of CHI 2025*.
  24. OSGuard: A Benchmark for Safety in Computer-Use Agents. *arXiv preprint arXiv:2606.15034*, 2026.
  25. Position: Episodic Memory is the Missing Piece for Long-Term LLM Agents. *arXiv preprint arXiv:2502.06975*, 2025.
  26. Precision Proactivity: Measuring Cognitive Load in Real-World AI-Assisted Work. *arXiv preprint arXiv:2505.10742*, 2025.

27. The Impact of LLM-Assistants on Software Developer Productivity: A Systematic Review. *arXiv preprint arXiv:2507.03156*, 2025.
28. UF02: The Desktop AgentOS. *arXiv preprint arXiv:2504.14603*, 2025.
29. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*.